

Introdução

Padrões de Projeto - Aula #01

Prof. Dr. Carlos Melo

 carlos.melo@upe.br



Sobre a disciplina

Objetivos

- Conhecer os diversos tipos de padrão de projeto de software.
- Analisar a aplicabilidade de cada padrão.
- Desenvolver sistemas de software utilizando padrões de projeto.



Sobre a disciplina

Metodologia e Avaliação

- Aprendizagem ativa
 - *vocês serão os protagonistas no próprio aprendizado*
- Baseada em Projetos
 - *mão na massa*
- Avaliação contínua por meio de:
 - *Listas de exercícios (10%);*
 - *Seminários (10%);*
 - *Projetos (30%);*
 - *Provas (50%).*

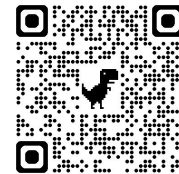


Sobre a disciplina

Presencial

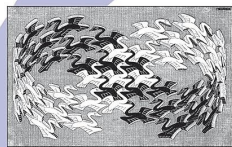
- 4 aulas por semana
 - sempre às quartas-feiras
- Teremos chamada
 - **presença obrigatória** , ou seja, **4 dias de falta** ...
- Teremos Monitoria (*dois dias por semana*)

Bibliografia



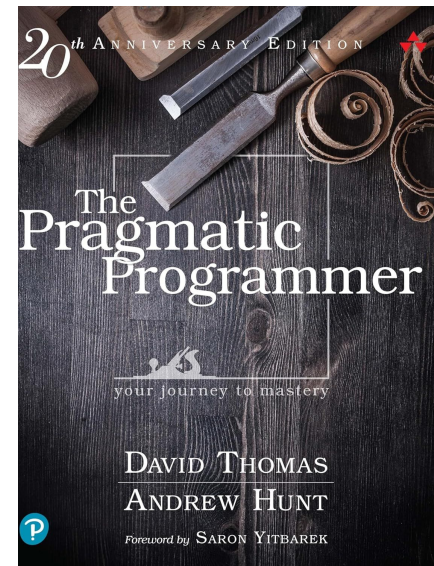
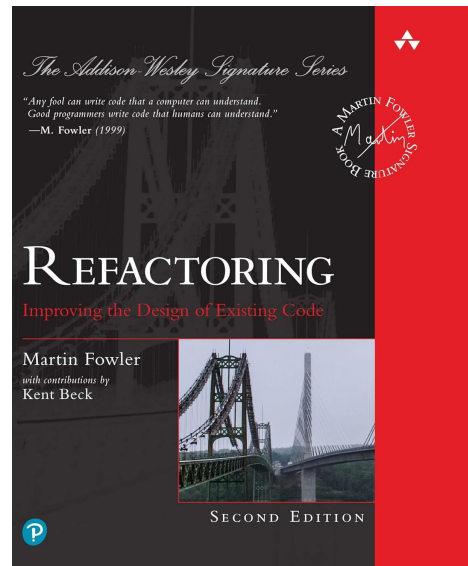
Padrões de Projeto

Soluções reutilizáveis de software orientado a objetos



ERICH GAMMA
RICHARD HELM
RALPH JOHNSON
JOHN VLISSIDES

Design Patterns



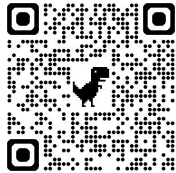


O problema

- Quem já se deparou com algum sistema de grande porte?
- Quem já precisou modificar código de outra pessoa?
- Quem já teve medo de alterar um sistema ou função?

O problema não é programar, o problema é manter o software.





Pensa comigo

```
1 def calcular_desconto(cliente):  
2     if cliente.tipo == "ouro":  
3         return 0.25  
4  
5     elif cliente.tipo == "prata":  
6         return 0.15  
7  
8     elif cliente.tipo == "bronze":  
9         return 0.05  
10  
11     else:  
12         return 0
```

- E se amanhã surgir:
 - Cliente Platinum?
 - Cliente Black?
 - Cliente Empresa?
 - Cliente Governo?
- O que acontecerá com essa função?



Pensa comigo

- Esse é o nosso problema na Engenharia de Software...
- Mas e as outras engenharias, como resolvem seus problemas?
 -  Engenharia Civil reutiliza projetos estruturais.
 -  Engenharia Aeronáutica reutiliza soluções certificadas.
 -  Engenharia Mecânica reutiliza componentes padronizados.
 -  Engenharia de Software reutiliza padrões de projeto.
- Não resolvemos cada problema do zero.
- Reutilizamos soluções que já funcionaram
 - somos preguiçosos mesmo.



Um pouco de História

- Na década de 1960–70, o arquiteto Christopher Alexander observou que **bons** arquitetos não projetavam cidades e edifícios do zero.
- Em vez disso, reutilizavam soluções recorrentes para problemas comuns: posicionamento de portas, janelas, praças e ruas.



Christopher
Alexander
1936 - 2022



Um pouco de História

- Ele registrou essas soluções em uma linguagem de padrões (Pattern Language), permitindo que arquitetos compartilhassem conhecimento e construíssem projetos de forma consistente.
 - Notes on the Synthesis of Form (1964)
 - A Pattern Language (1977): 253 padrões catalogados
 - The Timeless Way of Building (1979)
- Essa ideia inspirou diretamente a comunidade de Engenharia de Software anos depois.

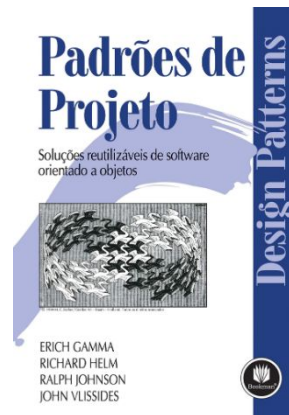


Um pouco de história

- No início dos anos 1990, quatro pesquisadores começaram a catalogar as soluções que se repetiam em grandes projetos de software.
- O resultado foi publicado em 1994:
 - Design Patterns: Elements of Reusable Object-Oriented Software.
- Eles identificaram 23 soluções recorrentes em projetos orientados a objetos.



A Gangue dos Quatro nos anos 1990'
carlos.melo@upe.br





O que é um padrão de projeto?

- Não é uma biblioteca.
- Não é um framework.
- Não é uma implementação pronta.
- É uma **solução** reutilizável para um problema recorrente:
 - Descreve como organizar classes e objetos para resolver um determinado problema de projeto de maneira reutilizável.

Os quatro elementos de um padrão de projeto



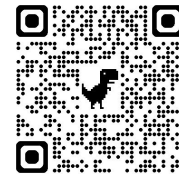
- Segundo o GoF, qualquer padrão de projeto pode ser descrito por quatro elementos fundamentais:
 - **Nome**
 - Permite que desenvolvedores conversem utilizando um vocabulário comum
 - **Problema**
 - Em qual situação esse padrão deve ser utilizado?
 - **Solução**
 - Como organizar objetos e classes para resolver esse problema?
 - **Consequência**
 - Quais as vantagens, limitações e impactos dessa decisão?



O Catálogo

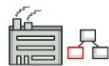
- O livro do GoF apresenta 23 padrões clássicos, organizados em 3 categorias por propósito:
 - Criacionais
 - Criação de objetos
 - Estruturais
 - Organização de Classes e Objetos
 - Comportamentais
 - Comunicação entre objetos
- Catálogos modernos (refactoring.guru) costumam listar 22, deixando de fora o Interpreter, raramente utilizado na prática.

O Catálogo dos Padrões de Projeto

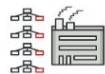


Padrões criacionais

Estes padrões fornecem vários mecanismos de criação de objetos, que aumentam a flexibilidade e reutilização de código já existente.



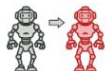
Factory Method



Abstract Factory



Builder



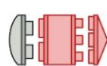
Prototype



Singleton

Padrões estruturais

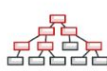
Estes padrões explicam como montar objetos e classes em estruturas maiores mas ainda mantendo essas estruturas flexíveis e eficientes.



Adapter



Bridge



Composite



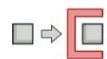
Decorator



Facade



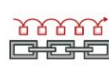
Flyweight



Proxy

Padrões comportamentais

Estes padrões são voltados aos algoritmos e a designação de responsabilidades entre objetos.



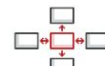
Chain of Responsibility



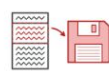
Command



Iterator



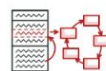
Mediator



Memento



Observer



State



Strategy



Template Method



Visitor



Por que precisamos de padrões de projeto?

- Poucos (nenhum) sistema permanece igual após serem entregues
- Um software sofre alterações para:
 - Correção de defeitos;
 - Adicionar novas funcionalidades;
 - Adaptar-se a novas tecnologias;
 - Atender novos requisitos;
 - Melhorar desempenho e segurança.

é melhor modificar um sistema com 500 linhas bem organizadas ou outros com 100 linhas fortemente acopladas?



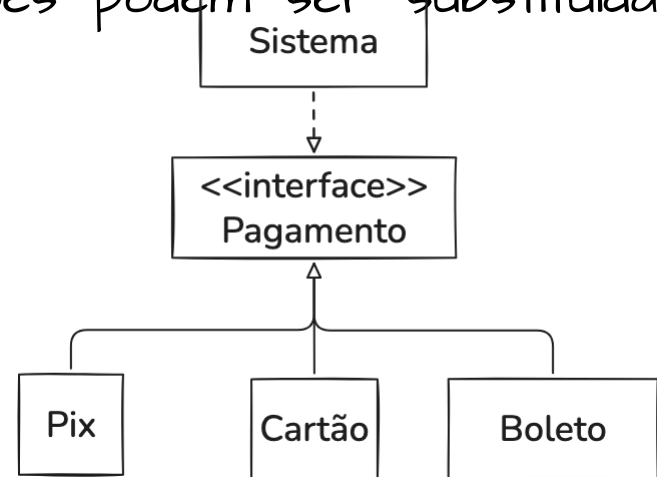
O custo da mudança

- Toda alteração tem um preço:
 - Introduzir novos bugs;
 - Quebrar funcionalidades existentes;
 - Aumento do acoplamento;
 - Dificultar testes;
 - Tornar o código menos compreensível.
- *É muito mais fácil mover o sofá de lugar que quebrar uma parede estrutural...*



Princípios

- #1 - Não dependa de implementações concretas
 - GoF: Programe para uma **interface**, não para uma implementação
 - O código deve depender de abstrações e não de classes específicas.
- Desse modo, diferentes implementações podem ser substituídas sem alterar o resto do sistema.
- Hoje:
 - **Pix()**
- Amanhã:
 - **Pix()**
 - **Cartao()**
 - **Boleto()**





Alto acoplamento

- Dependência de classe concreta

```
1 class Pix:
2     def pagar(self, valor):
3         print(f"PIX: R$ {valor}")
4
5 class Pedido:
6     def __init__(self):
7         self.pagamento = Pix()
8
9     def finalizar(self, valor):
10        self.pagamento.pagar(valor)
```

- O que acontece se quisermos aceitar cartão ou boleto?
 - Modificar Pedido;
 - Recompilar (em linguagens compiladas)
 - Aumento de acoplamento
 - Reimplementação de testes unitários



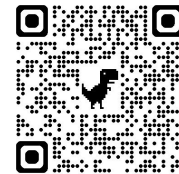
Alto acoplamento

- O problema não é o Pix, mas sim o fato de Pedido conhecer mais do que deveria
- Quem deveria decidir qual forma de pagamento será utilizada?

```
1 from abc import ABC, abstractmethod
2
3
4 class Pagamento(ABC):
5     @abstractmethod
6     def pagar(self, valor):
7         pass
```

```
1 class Pix(Pagamento):
2     def pagar(self, valor):
3         print(f"PIX: R$ {valor}")
4
5 class Cartao(Pagamento):
6     def pagar(self, valor):
7         print(f"Cartão: R$ {valor}")
```

- Todas as formas de pagamento agora tem o mesmo contrato, o resto do sistema só precisa conhecer pagamento, e não Pix ou Cartão, etc.



Quem decide?

- Antes:

```
1 class Pedido:
2     def __init__(self):
3         self.pagamento = Pix() # ⚠️ chumbado
4
5     def finalizar(self, valor):
6         self.pagamento.pagar(valor)
```

- Depois - quem chama escolhe o que chamar

```
1 class Pedido:
2     def __init__(self, pagamento: Pagamento): # Flexível
3         self.pagamento = pagamento
4
5     def finalizar(self, valor):
6         self.pagamento.pagar(valor)
7
8
9 # Não preciso mexer em pedido só passar o pagamento
10 Pedido(Pix()).finalizar(100)
11 Pedido(Cartao()).finalizar(100)
12 Pedido(Boleto()).finalizar(100)
```



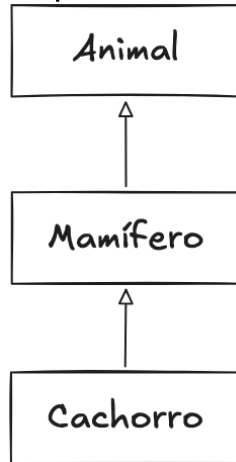
Vantagens de um baixo acoplamento

- Quando trabalhamos com abstrações:
 - é mais fácil e barato adicionar novas funcionalidades
 - facilita a execução de testes automatizados
 - melhora o reuso de componentes
 - aumenta a flexibilidade do sistema



Princípios

- #2 - Favoreça composição em vez de herança
 - Uso excessivo de herança pode aumentar o acoplamento e dificultar a evolução do sistema e ocorre em tempo de compilação
 - Objetos construídos por meio de composição são mais flexíveis, uma vez que são construídos em tempo de execução





Quando a herança é a escolha certa

- A herança nem sempre representa uma boa modelagem
 - Herança significa que algo É UM de alguma coisa
 - Precisa existir uma hierarquia entre as classes
 - Exemplo:

```
1 class Veiculo:
2     def __init__(self, placa): self.placa = placa
3
4 class Carro(Veiculo): ...
5 class Moto(Veiculo): ...
6 class Caminhao(Veiculo): ...
```



Quando a herança **não** é a escolha certa

- O é um que não é um

```
1 class Ave:
2     def voar(self):
3         print("voando")
4
5 class Pinguim(Ave):
6     def voar(self):
7         raise NotImplementedError("pinguim não voa")
```



O problema da herança

- Suponha que uma aplicação precise enviar notificações por diferentes canais:
 - e-mail, SMS, WhatsApp, Insta, ...
- Agora imagine que cada uma dessas notificações possa ser:
 - normal, criptografada, registrada em log, prioritária
- Como modelar isso apenas com Herança?
 - emailNormal, emailCriptografado, emailEmLog, emailPrioritario
 - ...



O problema da herança

- Quantas classes eu precisaria criar?
 - Comportamentos exclusivos 4 canais x 4 modos (16 classes)
 - Comportamentos combináveis (criptografada e em log e prioritária) 4 canais x 2^3 (32 classes)



O problema da herança

- Combinar objetos seria bem mais proveitoso (ainda temos muito o que fazer aqui, mas já é um avanço)



```
1 from abc import ABC, abstractmethod
2
3 class CanalNotificacao(ABC):
4     @abstractmethod
5     def enviar(self, mensagem):
6         pass
```



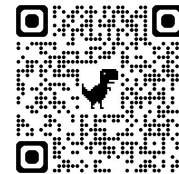
```
1 class Email(CanalNotificacao):
2     def enviar(self, mensagem):
3         print(f"E-mail: {mensagem}")
4
5
6 class SMS(CanalNotificacao):
7     def enviar(self, mensagem):
8         print(f"SMS: {mensagem}")
9
10 ...
```



```
1 class Notificacao:
2     def __init__(self, canal):
3         self.canal = canal
4
5     def enviar(self, mensagem):
6         self.canal.enviar(mensagem)
```



```
1 notificacao = Notificacao(Email())
2 notificacao.enviar("Olá!")
3
4 notificacao = Notificacao(SMS())
5 notificacao.enviar("Olá!")
```



Prof. Dr. Carlos Melo

<https://calendly.com/prof-casm>

 carlos.melo@upe.br